# Clasificador de imágenes inmobiliarias

**Autores:** Pedro Calderón, Juan Miguel Correa, Miguel Mota Cava **Asignatura:** Machine Learning II — Máster en Big Data, curso 2025/26 **Repositorio:** https://github.com/Miguelmotacava/real-estate-image-classifier **Workspace W&B:** https://wandb.ai/jumipe_meflipapesos/real-estate-classifier

## 1. Contexto de negocio

El cliente es un marketplace inmobiliario online. Cada anuncio lleva varias fotos subidas por el anunciante (salón, cocina, dormitorio, fachada, calle, etc.) y hoy se etiquetan a mano por el equipo de operaciones. Con volúmenes del orden de miles de fotos al día, el coste es alto y la consistencia baja: un mismo "salón con cocina americana" puede quedar clasificado como salón o como cocina según quién lo haya revisado. Esa inconsistencia se nota en el ranking del buscador y en los filtros de la ficha del inmueble.

Lo que se entrega es una API que clasifica automáticamente cada imagen dentro de 15 clases del dataset 15-Scene, mapeadas a tres familias que sí se usan en el producto: Interior (Bedroom, Kitchen, Living room, Office, Store, Industrial), Exterior urbano (Inside city, Tall building, Street, Suburb, Highway) y Entorno natural (Coast, Forest, Mountain, Open country). El caller principal es el equipo de producto: la categorización alimenta filtros de búsqueda, detección de anuncios incompletos (falta foto de baño, por ejemplo) y deduplicación visual.

Las restricciones duras acordadas con el negocio son dos: latencia ≤1 s en el flujo síncrono del subir-anuncio y ≤5 s en revisiones masivas tipo backfill, y trazabilidad completa de cada predicción (qué modelo la hizo, en qué dispositivo y en cuánto tiempo) para auditoría.

## 2. Arquitectura del sistema

Hay tres bloques: training, servicio y tracking.

```
fotos (JPG/PNG) → Streamlit (8501) → FastAPI /predict (8000) → PyTorch (timm) → respue
                                              │
                                              └── ModelRegistry (5 modelos seleccionabl

training/ (src/experiments/*)  ──────────►  W&B (runs, métricas, figuras, artifacts)

docker compose: api + streamlit en la misma red, models/ montado como volumen
```

La API expone cuatro endpoints y puede servir distintos modelos según el parámetro `?model=` en la query. Por defecto arranca con `FINAL` (un único backbone Swin-Large 384), pensado para el caso síncrono, y ofrece `ensemble` como modo de máxima accuracy para el caso batch o para auditoría. Los 4 miembros individuales también son accesibles uno por uno, algo útil para comparar predicciones cuando hay dudas.

## 3. Aproximación al modelado

### 3.1 Partición del dataset

El dataset 15-Scene contiene 4485 imágenes en 15 clases. Se dividió dos veces:

- **70/15/15 estratificado con seed=42** para toda la fase de experimentación (comparar arquitecturas, benchmarks contra un test no visto).
- **90/10 estratificado con seed=42** para el modelo que realmente se despliega, una vez elegido el

approach. La idea es simple: el test holdout ya cumplió su función en la fase de experimentación, así que en la versión final se recupera ese 15% como train adicional para darle al modelo el máximo de datos posible.

### 3.2 Backbones elegidos

La receta final combina cuatro backbones preentrenados distintos para tener diversidad arquitectónica en el ensemble:

| Backbone | Parámetros | Pretraining |
|---|---|---|
| Swin-Large 384 (`swin_large_patch4_window12_384.ms_in22k_ft_in1k`) | 197 M | IN22K → IN1K supervisado |
| ConvNeXtV2-Large 288 (`convnextv2_large.fcmae_ft_in22k_in1k`) | 198 M | FCMAE + IN22K/IN1K |
| EVA02-Base 448 (`eva02_base_patch14_448.mim_in22k_ft_in22k_in1k`) | 87 M | MIM + IN22K |
| BEiT-Large 224 (`beit_large_patch16_224.in22k_ft_in22k_in1k`) | 303 M | MIM clásico + IN22K |

La decisión fue deliberada: un CNN moderno (ConvNeXtV2), un ViT con masked image modeling (EVA02), un transformer jerárquico con ventanas (Swin) y otro transformer con MIM clásico (BEiT). Individualmente los cuatro rondan val ~0.98, pero sus errores están decorrelacionados, que es lo que necesita un ensemble para mejorar.

### 3.3 Transfer learning

Se usó fine-tuning diferencial en los cuatro modelos: el clasificador (la cabeza) entrena con `lr = 7e-4`, y el backbone entrena con `lr = 7e-4 × 0.05 = 3.5e-5`. Es el compromiso clásico para que la cabeza se ajuste rápido sin perturbar demasiado las representaciones preentrenadas de ImageNet.

El resto de la receta es estándar pero se fue refinando a lo largo de los bloques A→F: `CrossEntropyLoss` con `label_smoothing=0.1`, optimizador `AdamW` con `weight_decay=1e-4`, cosine LR annealing con 2 epochs de linear warmup, mixed precision (AMP) en GPU y EMA de pesos con decay 0.999. Al final de cada epoch comparamos `val_acc` del modelo "raw" y del modelo EMA, y nos quedamos con el mejor para el checkpoint.

Las augmentations de train son moderadas: `RandomResizedCrop(0.7-1.0)`, hflip, rotación ±15°, ColorJitter ligero y `RandomErasing(p=0.25)`. Se descartó MixUp y CutMix porque en el bloque C se observó que, con esta receta de regularización (`drop_path=0.1`, weight decay, label smoothing), el overfit ya estaba controlado y la mezcla de imágenes rompía categorías espacialmente sensibles como Kitchen/Bedroom.

### 3.4 Arquitectura final que sirve la API

El modelo campeón es un ensemble 4-way soft-voting sobre los cuatro backbones, con tres capas de TTA:

1. Cada miembro procesa la imagen a **3 escalas** (0.82, 0.88, 0.94 del tamaño nativo del modelo).
2. Por cada escala, **5 crops** (centro + 4 esquinas).
3. Por cada crop, **2 flips** (original + hflip).

Eso da 30 vistas por imagen y por modelo, 120 vistas totales. Se promedian las softmax de cada vista dentro de cada modelo y luego se combinan los 4 resultados con pesos elegidos por grid search sobre el

simplex 4D (paso 0.05, 1771 puntos evaluados) maximizando val_accuracy en las 449 imágenes del split 90/10. Los pesos óptimos son:

```
0.15 · ConvNeXtV2-L   +   0.60 · EVA02-B 448   +   0.25 · Swin-L 384   +   0.00 · BEiT-L
```

Que BEiT quedara a peso 0 fue una sorpresa: individualmente acertaba val 0.9755, pero sus errores solapaban demasiado con los del resto. El grid search lo descartó automáticamente. Aun así, entrenarlo merecía la pena: el propio procedimiento de selección tiene que verlo para poder rechazarlo, y el miembro queda disponible si en el futuro se añaden datos o se sustituye.

## 4. Proceso de experimentación

Todo el tracking está en el proyecto W&B `real-estate-classifier`. El trabajo se organizó en bloques, cada uno cerrando una pregunta antes de pasar al siguiente.

**Bloque A — viabilidad (CPU).** Primeros runs sin GPU, imagen 160×160, 1600 imágenes de train. Sirvió para verificar que feature-extraction pura no llega (val ~0.32) y que hay que hacer fine-tuning. Mejor resultado: MobileNetV3-Small en CPU, val 0.714.

**Bloque B — screening de backbones (GPU, 3 epochs, 224×224).** Se entrenaron seis arquitecturas diferentes (EfficientNet-B0/B3, ResNet-50, RegNetY, ConvNeXt-Tiny, DeiT-S) con la misma receta para ver cuál respondía mejor a pocos epochs. Ganador: ConvNeXt-Tiny, val 0.945. Los EfficientNet quedaron últimos porque su pretraining es más antiguo.

**Bloque C — receta anti-overfit.** Sobre ConvNeXt-Tiny, se subió el régimen de regularización: `drop_path=0.2`, `weight_decay` más agresivo, `label_smoothing=0.15`, TrivialAugmentWide y RandomErasing 0.4. El resultado fue contraintuitivo pero sano: `train_acc` bajó a 0.959 (antes estaba en 0.99+ sobreajustando), pero val y test subieron a 0.967 y 0.975 respectivamente. El gap se invirtió, señal de que el modelo generaliza.

**Bloque D — primeros ensembles sin TTA.** Soft-voting de varios ConvNeXt + DeiT. No mejoraron al modelo individual C2 porque los miembros eran demasiado parecidos entre sí.

**Bloque E — escalar backbones.** Se entrenaron ConvNeXt-Small y ConvNeXt-Base a 288×288. El Base individual alcanzó val 0.982, test 0.973. Combinado con el Tiny regularizado y TTA de 5-crop selectivo llegamos a test 0.987 (E5), que fue el mejor ensemble antes de subir de nivel de backbone.

**Bloque F — backbones Large + multi-scale TTA.** Se entrenaron ConvNeXtV2-L (F3), EVA02-B 448 (F4), Swin-L 384 (F6) y BEiT-L (F8). Cada uno individualmente se clava en val 0.975-0.979, que es esencialmente el techo irreducible del dataset para un modelo solo (hay aproximadamente 14 imágenes que ningún modelo acierta). La diversidad arquitectónica es lo que rompe ese techo: el ensemble F9 (F3+F4+F6+F8 con TTA 30-view) llega a test 0.9896 y F1 macro 0.9908 sobre el split 70/15/15.

**Bloque FINAL (90/10).** Con la receta ya validada y la arquitectura elegida, se reentrenaron en 90/10 los cuatro miembros y se corrió el grid search del ensemble sobre esas 449 imágenes de val. El resultado es el que sirve ahora la API: **val 0.9933 y F1 macro 0.9938**.

Para hyperparameter search se combinaron tres estrategias:

- **Sweep bayesiano** (`src/experiments/sweep.py`) en el bloque C sobre lr, weight_decay, label_smoothing y dropout. Cuatro runs paralelos con early stopping.
- **Grid search en simplex** para los pesos del ensemble, paso 0.05. Se maximizó val_accuracy en lugar de test para no introducir fuga de información.
- **Transferencia de recipes**: una vez validada la receta de C2, se adaptó a cada Large cambiando

sólo `lr` y `batch_size` según capacidad de la GPU.

No todo salió al primer intento. Dos incidencias merece la pena contar porque explican decisiones de este informe:

- En el primer reentrenamiento del Swin-L 384 con split 90/10 la GPU hizo thermal throttling severo (86 °C sostenidos, epoch 5751 s en lugar de ~300 s). Solamente dio tiempo a completar 1 epoch antes de pararlo. El checkpoint de ese epoch ya daba val 0.978, suficiente para entregar, pero se relanzó una vez la GPU se enfrió y ahí sí llegó a val 0.9866 en 8 epochs.
- F10 DeiT-III Large 384 fue el candidato a quinto miembro, pero entrenarlo fue impracticable por thermals similares, 48 min/epoch. Se abortó en epoch 6 y no se incorporó. Queda documentado como "cosa que se probó y se descartó", no como un modelo entrenado a medias.

## 5. Métricas por clase

Las cifras de abajo son las del ensemble final sobre las 449 imágenes de val del split 90/10. Se ordenan por F1 descendente:

| Clase | Etiqueta negocio | Precision | Recall | F1 | N |
|---|---|---|---|---|---|
| Bedroom | Dormitorio | 1.000 | 1.000 | 1.000 | 22 |
| Coast | Costa / Mar | 1.000 | 1.000 | 1.000 | 36 |
| Forest | Bosque | 1.000 | 1.000 | 1.000 | 33 |
| Highway | Carretera | 1.000 | 1.000 | 1.000 | 26 |
| Industrial | Nave industrial | 1.000 | 1.000 | 1.000 | 31 |
| Kitchen | Cocina | 1.000 | 1.000 | 1.000 | 21 |
| Living room | Salón | 1.000 | 1.000 | 1.000 | 29 |
| Mountain | Montaña | 1.000 | 1.000 | 1.000 | 37 |
| Office | Despacho | 1.000 | 1.000 | 1.000 | 21 |
| Store | Local comercial | 1.000 | 1.000 | 1.000 | 32 |
| Suburb | Suburbio | 1.000 | 1.000 | 1.000 | 24 |
| Tall building | Edificio | 1.000 | 0.981 | 0.991 | 54 |
| Street | Calle | 0.967 | 1.000 | 0.983 | 29 |
| Open country | Campo abierto | 0.983 | 0.976 | 0.979 | 41 |
| Inside city | Vista urbana | 1.000 | 0.935 | 0.967 | 31 |
| **macro avg** | | **0.997** | **0.992** | **0.994** | 449 |

Once clases salen sin un solo error, incluidas todas las que el negocio considera "altas apuestas" (interiores del inmueble y fachadas). Las tres clases con F1 por debajo de 0.99 son las que ya en el Bloque F dimos por imposibles de resolver con este dataset: Inside city, Street y Open country. Son ambiguas incluso para personas (una vista aérea sin horizonte entre campo y costa, o una calle estrecha con fachadas a ambos lados que podría ser Street o Inside city), y el análisis de errores mostró que los tres o cuatro fallos residuales están concentrados en esas dos fronteras. Para el caso de uso del marketplace eso no es crítico: esos tipos se usan para filtros de búsqueda, no para el ranking principal.

La interpretación de la matriz de confusión confirma lo anterior: los errores no son aleatorios ni cruzan familias (nunca una Kitchen acaba clasificada como Street), sólo aparecen en las dos fronteras citadas dentro de Exterior urbano y Entorno natural.

## 6. Documentación de la API

La API está implementada en FastAPI y expone OpenAPI 3.1 automáticamente. Swagger UI está en `/docs` y ReDoc en `/redoc`. Los endpoints:

| Método | Ruta | Descripción |
|---|---|---|
| GET | `/health` | estado del servicio y modelo cargado por defecto |
| GET | `/classes` | catálogo de 15 clases con el mapeo al vocabulario de negocio |
| GET | `/models` | lista de modelos seleccionables vía `?model=<alias>` |
| POST | `/predict?model=<alias>` | clasifica una imagen (JPG/PNG/WEBP, ≤8 MB) |
| POST | `/predict/batch?model=<alias>` | clasifica varias imágenes en una sola llamada |

Los cinco alias admitidos son `FINAL` (default), `F3`, `F4`, `F8` y `ensemble`. Llamar a `/predict` sin el parámetro `model` usa `FINAL`. Ejemplo de respuesta:

```
{
  "filename": "salon_002.jpg",
  "class": "Living room",
  "business_label": "Salón",
  "confidence": 0.987,
  "top3": [
    {"label": "Living room", "business_label": "Salón", "confidence": 0.987},
    {"label": "Bedroom", "business_label": "Dormitorio", "confidence": 0.008},
    {"label": "Office", "business_label": "Despacho", "confidence": 0.003}
  ],
  "model_used": "ensemble[F3,F4,FINAL,F8]",
  "model_alias": "ensemble",
  "inference_device": "cuda",
  "inference_time_ms": 4217.6
}
```

El error handling está dividido en dos capas. Validación del upload (tipo MIME y tamaño) en un helper, con códigos 413 y 415 según corresponda. Validación de contenido (imagen corrupta) dentro de `predict_image`, con 400. Si se pide un modelo que no existe en el registro se devuelve 404 con el alias recibido en el detail. El endpoint batch nunca falla globalmente: si una imagen entre diez es inválida, las otras nueve siguen procesándose y la inválida aparece en `errors[]`.

La UI de Streamlit es un front fino sobre la API (no tiene lógica de modelo propia). Tiene un desplegable en la sidebar para elegir modelo, un área de drag-and-drop para la imagen, y muestra la categoría predicha, el top-3 con barras de confianza y un panel expandible con detalles técnicos (modelo, dispositivo, tiempo).

## 7. Conclusiones y recomendaciones

1. **Para el flujo síncrono del marketplace, desplegar `FINAL` por defecto.** Un único Swin-L 384, aproximadamente 90 ms por imagen en GPU, val 0.9866, F1 macro 0.988. La API ya arranca con él y ningún cliente tiene que hacer nada especial.

2. **Activar `ensemble` como "modo auditoría" en backfills y revisiones premium.** Gana 0.7 puntos de accuracy (0.9933) pero paga unas 45× más de latencia (~4 s). Para re-etiquetar el histórico del catálogo de una vez es asumible; para el flujo en vivo, no.

3. **Si se añade un feedback loop**, la ganancia futura está en datos reales del marketplace, no en más capacidad de modelo. Las dos clases con F1 < 0.99 son justamente las que más se beneficiarían de fotos del inventario real (vistas de calles urbanas estrechas, fotos aéreas) que no existen en 15-Scene. Se recomienda añadir un endpoint `POST /feedback` donde el operador corrija clases erróneas y reentrenar incrementalmente una vez al mes.

4. **Trazabilidad por defecto.** Cada predicción devuelve `model_used`, `model_alias`, `inference_device` e `inference_time_ms`. El equipo de producto puede auditar "¿qué modelo etiquetó este anuncio?" sin añadir logging adicional.

5. **Lección general del proyecto:** con un dataset de ~4500 imágenes, escalar el tamaño del backbone tiene rendimientos decrecientes a partir de cierto punto. Lo que realmente mueve la aguja es la diversidad arquitectónica del ensemble. Cuatro backbones Large que usan paradigmas distintos (CNN, ViT con MIM, Swin jerárquico, BEiT-MIM) llegan combinados a un sitio donde ninguno llega solo.

## 8. Enlaces y acceso

Repositorio público: https://github.com/Miguelmotacava/real-estate-image-classifier

W&B workspace: https://wandb.ai/jumipe_meflipapesos/real-estate-classifier

Los cinco modelos están en W&B como artifacts versionados bajo los nombres `model-exp_FINAL_swin_large_384_9010`, `model-exp_FINAL_F3_9010`, `model-exp_FINAL_F4_9010`, `model-exp_FINAL_F8_9010` y `model-exp_FINAL_ensemble_9010`. El README del repo incluye las instrucciones para descargarlos desde `wandb artifact get`.

`agascon@comillas.edu` y `rkramer@comillas.edu` están invitados al workspace de W&B con permisos de viewer. Ver `reports/wandb_invite.md` para la captura del envío.